System Design Interview
Jan Lunenborg

Foreword
Join the Community
Scale from Zero To Millions of Users
Back of the envelope Estimation
A Framework for System Design Interviews
Design A Rate Limiter
Design Consistent Hashing
Design A Key-value Store
Design A Unique ID Generator In Distributed Systems
Design A URL Shortener
Design A Web Crawler
Design A Notification System
Design A News Feed System
Design A Chat System
Design A Search Autocomplete System
Design Youtube
Design Google Drive
Proximity Service
Nearby Friends
Google Maps
Distributed Message Queue
Metrics Monitoring and Alerting System
Ad Click Event Aggregation
Hotel Reservation System
Distributed Email Service
S3-like Object Storage
Real time Gaming Leaderboard
Payment System
Digital Wallet
Stock Exchange
The Learning Continues

14
Design A Search Autocomplete System

When searching on Google or shopping at Amazon, as you type in the search box, one or more matches for the search term are presented to you. This feature is called search autocomplete, typeahead, search as you type, or incremental search. It is the most common feature in all search engines and shows a list of autocomplete recommendations under "Enter" a typed into the search box. Search autocomplete is a fundamental feature of many products. In this chapter, we will design a search autocomplete system, also called "design typeahead" or "design type-to-read-ahead" system.

Figure 1

Step 1 - Understand the problem and establish design scope

The first step is to clarify design intended questions to ask candidate and to clarify requirements. Here is an example of candidate-interviewer interaction:

Candidate: Is the matching only supported at the beginning of a search query or in the middle as well?
Interviewer: It's at the beginning of a search query.

Candidate: How many autocomplete suggestions should the system return?
Interviewer: Five.

Candidate: How do the system know which it suggestions to return?
Interviewer: This is determined by popularity, decided by the historical query frequency.

Candidate: Does the system support spell check?
Interviewer: No, spell check or autocorrect is not supported.

Candidate: Are search queries in English?
Interviewer: Yes, English search queries only, we can discuss multi language support.

Candidate: Is any advanced capitalization and special characters?
Interviewer: No, we assume all search queries have lowercase alphabetic characters.

Candidate: How many users use the product?
Interviewer: 10 million (DAU)

Requirements

Here is a summary of the requirements:

- Fast response time: As you type a search query, autocomplete suggestions must show up fast enough. An article about Facebook typeahead system suggests the system should return results within 100 milliseconds. Otherwise it will cause stuttering.
- Relevant: Autocomplete suggestions should be relevant to the search term.
- Sorted: Results returned by the system must be sorted by popularity or other ranking models.
- Scalable: The system can handle traffic volume.
- High availability: The system should remain available and accessible when part of the system is offline, slow down, or experiences unexpected network errors.

Back of the envelope estimation

- Assume 10 million daily active users (DAU)
- On average person perform 10 searches per day
- 20 bytes of data per query string
- Assume English characters are stored using 1 character = 1 byte
- Assume we store 10 years worth of query strings in our system
- Traffic is 2:1 (10 daily active users = 20 queries per second)
- For every character entered into the search box, a client sends a request to the backend for autocomplete suggestions. On average, 20 requests are sent for each search query. For example, the following 6 characters sent to the backend by the time you finish typing your search query: "dresses".

dr
dre
dres
dress
dresses

- Peak QPS = 20 * 2 = 40
- To get top 5 frequently searched queries, execute the following SQL query:

```
SELECT * FROM frequency_table
WHERE query LIKE 'prefix%'
ORDER BY frequency DESC
LIMIT 5
```

Figure 4

This is an acceptable solution when the system is small. However, when it is large, accessing the database becomes a bottleneck. We will explore optimizations in later time.

Step 2 - Design high-level design and

At the high level design, we discussed data gathering service and query service. The high level design is not optimal, but it serves as a good starting point. In this section, we will dive deep into a few components and explore optimizations as follows:

- Trie data structure
- Data gathering service
- Query service
- Scale the storage
- Trie operations

Trie data structure

Relational databases are used for storage in the high-level design. However, fetching the top 5 search queries from a relational database is slow if we have a large data set. As a rule of thumb, it takes a single server hundreds of GB of disk space to store a large table. Most database significant time to retrieve a row from disk, especially if the table size is larger than the size of the disk and from archives [2] and [3].

Understanding the trees: trie data structure is essential for this interview question. Fortunately, this is kind of a data structure question than a product design question. Besides, many online materials explain the concept. In this chapter, we will only discuss an overview of the trie data structure and focus on how to optimize the trie data to improve response time.

Trie (pronounced "try") is a tree-based data structure that can compactly store strings. The name comes from the word **retrieval**, which indicates it is designed for string retrieval operations. The main idea of the trie data structure is:

- A trie is a tree like data structure.
- The root represents an empty string.
- Each node stores a character and has 26 children, one for each possible character. To save space, we do not draw empty lines.
- Each tree node represents a single character and a possible character path.

Figure 5 shows a trie with search queries "tea", "to", "top", "toy", "toys", "tea". Search queries are highlighted with a thicker border.

Figure 5

Basic trie data structure stores characters in nodes. To support sorting by frequency, frequency info needs to be included in nodes. Assume we have the following frequency table.

query	frequency
tea	10
to	20
top	15
toy	12
toys	8
tea	10

Table 2

After adding frequency info to nodes, updated trie data structure is shown in Figure 6.

Figure 6

Have data accessible with unit time? Before diving into the algorithm, let us define some terms:

- a length of a prefix
- a total number of nodes in a trie
- a number of children of a given node

Trie is a good in most scenarios because it has two advantages:

1 Find the prefix. Time complexity: O(p)

2. Traverse the subtree from the prefix node to get all valid children. A trie is valid if it can form a valid query string. Time complexity: O(c)

Let us look at these optimizations one by one.

Let us use an example as shown in Figure 7 to explain the algorithm. Assume a request is sent to a cache server for "tea". The search node. The algorithm works as follows:

- Step 1: Find the prefix node "t"
- Step 2: Traverse the subtree to get all valid children. In this case, nodes [tea: 10], [toe: 30], [toy: 35] are valid.
- Step 3: Sort the children and get top 2. [toe: 30] and [toy: 35] are the top 2 queries both prefix "t".

Figure 7

The time complexity of the algorithm is the sum of time spent on each step mentioned above: **Step 1 (Step 2 + Step3)**

Trie allows algorithm is straightforward. However, it is too slow because we need to traverse the entire trie to get top 5 results in the worst case scenario. Below are two optimizations:

1 Limit the max length of prefix

2. Cache top search queries at each node

Let us look at these optimizations one by one.

Limit the max length of a prefix

Users rarely type a long search query into the search box. Thus, it is safe to say p is a small integer number, say 50. If we limit the length of a prefix, the number of nodes we must traverse to get the prefix node can be treated from O(p) to O(small constant), aka O(1).

Cache top search queries at each node

Prefix length is limited to a small number, and we know a finite frequently used queries at each node. Since 5 to 10 autocomplete suggestions are enough for serving 10 million daily active people, it can specifically cache the top 5 search queries at each node.

By caching top search queries at every node, we significantly reduce the time complexity to retrieve the top 5 queries. However, this design requires a lot of space to store top 5 queries at every node. Trading space for speed is a time-space tradeoff, a common design in computer.

Figure 8 shows the updated trie data structure. The queries are stored on each node. For example, the node with prefix "tea" stores the following: [tea: 10, tea: 20, tea: 30, tea: 15, tea: 10].

Figure 8

Analytics Logs: It stores raw data about search queries. Logs are appended only and not indexed. Table 3 shows an example of the log file.

query	time
tea	2018-10-31 22:01:01
tea	2018-10-31 22:01:05
tree	2018-10-31 22:01:10
tea	2018-10-31 22:02:22
tea	2018-10-31 22:02:41
toy	2018-10-31 22:03:03

Table 3

Aggregators: The size of analytics logs is usually very large, and data is not in the right format. We need to aggregate data so it can be easily consumed by other services.

Depending on the use case, we may aggregate data differently for realtime applications such as Twitter, we aggregate data in a shorter time interval as real-time results are important. On the other hand, aggregating data less frequently, say once per week, might be good enough for many use cases. Given the trie data structure, we'll assume that the trie data structure is recreated weekly.

Aggregated Data

Table 4 shows an example of aggregated weekly data "time" field represents the start time of a week. "Frequency" field is the sum of the occurrences for the corresponding query in that week.

query	time	frequency
tea	2018-10-21	12002
tea	2018-10-28	10800
tea	2018-10-14	8000
tea	2018-10-07	9000
tea	2018-10-30	8200
tea	2018-10-23	7800

Table 4

Workers: Workers are a set of servers that perform asynchronous jobs at regular intervals. They build the data structure and store it in the trie db.

Trie Cache: Trie Cache is a distributed cache system that keeps trie in memory for fast read. It takes a weekly snapshot of the DB.

Trie DB: Trie DB is the persistent storage. Two options are available for the trie db:

1 Document store. Since a new trie is built weekly, we can periodically take a snapshot of it, serialize it, and store the serialized data in persistent storage. Many document stores are suitable for this scenario.

2 Key-value store. A trie can be represented in a key-value form as follows:

- Every prefix in the trie is mapped to a key in a hash table.
- Data on each node is mapped to a value in a hash table.

Figure 10 shows the mapping between the trie and hash table.

Figure 10

In Figure 10, each trie node on the left is mapped to the key, value pair on the right. If you are stricter how key-value stores work, refer to that "Design a key-value store" chapter.

Query service

In the high-level design, query service calls the database directly to fetch the top 5 results. Figure 11 shows the improved design on previous design in bullet points.

Figure 11

1. A search query is sent to the load balancer.
2. The load balancer routes the request to API servers.
3. API servers get trie data from Trie Cache and construct autocomplete suggestions for the client.
4. In case the data is not in the cache, we fill the cache on a cache miss logic. This ensures all subsequent requests for the same prefix are returned from the cache. Cache may need to be refreshed if the cached data is stale.

Query service requires performing two steps. We propose the following optimizations:

1. Add request to the queue. Incoming queries would be 5,000 requests to fetch autocomplete results. The rate fetch/sec of 5,000 is too overwhelming a requestprocessor does not refresh the cache and page.
2. Browser caching. For many applications, autocomplete results suggestions may not change much within a short time. Thus, autocomplete suggestions can be stored in browser cache to allow subsequent autocomplete suggestions retrieval. For example, in the search application, suggestions may change every one hour. Setting browser cache to one hour is a good choice. Below are two optimizations:
   - Use "max-age" in the Cache-control header to specify how long a response should be cached. For example, Cache-control: max-age=3600 sets the Google search query time-to-live in the Response for "Cron" Response time "expires" in the Google search query time-to-live. Google caches the results in the Response for "Cron" Response time "expires" in the Google search query time-to

Since English is the only supported language, a naive way to shard is based on the first character. Here are some examples.

- If we need two servers for storage, we can store queries starting with *'a'* to *'m'* on the first server, and *'n'* to *'z'* on the second server.
- If we need three servers, we can split queries into *'a'* to *'i'*, *'j'* to *'r'* and *'s'* to *'z'*.

Following this logic, we can split queries up to 26 servers because there are 26 alphabetic characters in English. Let us define sharding based on the first character as first level sharding. To store data beyond 26 servers, we can shard on the second or even at the third level. For example, data queries that start with *'a'* can be split into 4 servers: *'aa-ag'*, *'ah-an'*, *'ao-au'*, and *'av-az'*.

At the first glance this approach seems reasonable, until you realize that there are a lot more words that start with the letter *'c'* than *'x'*. This creates uneven distribution.

To mitigate the data imbalance problem, we analyze historical data distribution pattern and apply smarter sharding logic as shown in Figure 15. The shard map manager maintains a lookup database for identifying where rows should be stored. For example, if there are a similar number of historical queries for *'s'* and for *'u'*, *'v'*, *'w'*, *'x'*, *'y'* and *'z'* combined, we can maintain two shards: one for 's' and one for 'u' to 'z'.

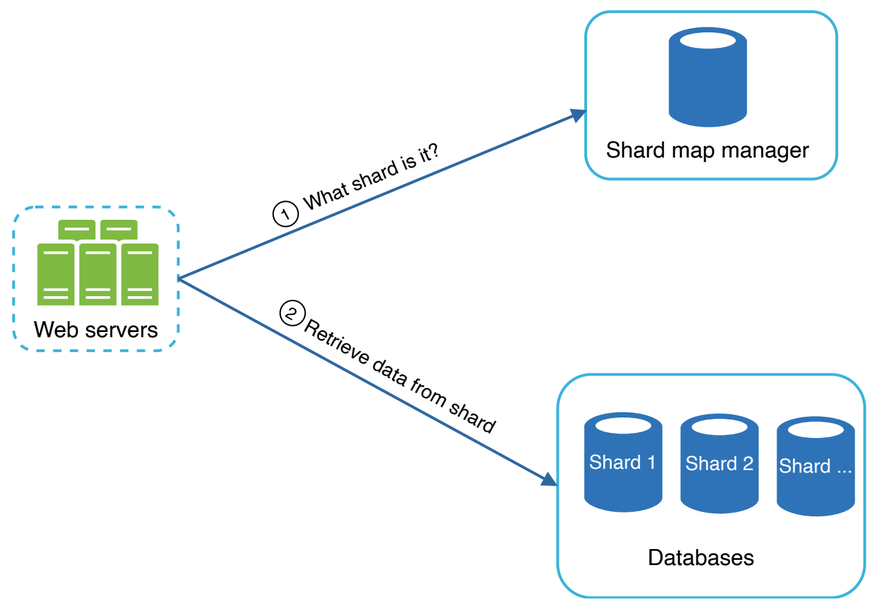

Figure 15

Step 4 - Wrap up

After you finish the deep dive, your interviewer might ask you some follow up questions.

Interviewer: How do you extend your design to support multiple languages?

To support other non-English queries, we store Unicode characters in trie nodes. If you are not familiar with Unicode, here is the definition: "an encoding standard covers all the characters for all the writing systems of the world, modern and ancient" [5].

Interviewer: What if top search queries in one country are different from others?

In this case, we might build different tries for different countries. To improve the response time, we can store tries in CDNs.

Interviewer: How can we support the trending (real-time) search queries?

Assuming a news event breaks out, a search query suddenly becomes popular. Our original design will not work because:

- Offline workers are not scheduled to update the trie yet because this is scheduled to run on weekly basis.
- Even if it is scheduled, it takes too long to build the trie.

Building a real-time search autocomplete is complicated and is beyond the scope of this course so we will only give a few ideas:

- Reduce the working data set by sharding.
- Change the ranking model and assign more weight to recent search queries.
- Data may come as streams, so we do not have access to all the data at once. Streaming data means data is generated continuously. Stream processing requires a different set of systems: Apache Hadoop MapReduce [6], Apache Spark Streaming [7], Apache Storm [8], Apache Kafka [9], etc. Because all those topics require specific domain knowledge, we are not going into detail here.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Reference materials

[1] The Life of a Typeahead Query:
https://www.facebook.com/notes/facebook-engineering/the-life-of-a-typeahead-query/389105248919/

[2] How We Built Prefixy: A Scalable Prefix Search Service for Powering Autocomplete:
https://medium.com/@prefixyteam/how-we-built-prefixy-a-scalable-prefix-search-service-for-powering-autocomplete-c20f98e2eff1

[3] Prefix Hash Tree An Indexing Data Structure over Distributed Hash Tables:
https://people.eecs.berkeley.edu/~sylvia/papers/pht.pdf

[4] MongoDB wikipedia: https://en.wikipedia.org/wiki/MongoDB

[5] Unicode frequently asked questions: https://www.unicode.org/faq/basic_q.html

[6] Apache hadoop: https://hadoop.apache.org/

[7] Spark streaming: https://spark.apache.org/streaming/

[8] Apache storm: https://storm.apache.org/

[9] Apache kafka: https://kafka.apache.org/documentation/

Mark as Complete

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.